

Laravel 13 Enums

An easy, practical guide — creating, validating, reading, and displaying enums



Draft

Pending Review

Active

Rejected

Archived

1. Create the Enum

Use the Artisan command with the folder path so it lands in `app/Enums/` :

```
$ php artisan make:enum Enums/ProductStatus --string
```

This generates `app/Enums/ProductStatus.php` . Fill in your cases:

```
<?php

namespace App\Enums;

enum ProductStatus: string
{
    case Draft = 'draft';
    case PendingReview = 'pending_review';
    case Active = 'active';
    case Rejected = 'rejected';
    case Archived = 'archived';
}
```

That's it — the enum exists. Each `case` has a name (e.g. `Draft`) and a value (`'draft'`) that matches what's stored in the database.

2. Add Helper Methods (Labels, Colors, etc.)

This is where enums become genuinely useful — you can attach behavior directly to each case using `match`:

```
<?php

namespace App\Enums;

enum ProductStatus: string
{
    case Draft = 'draft';
    case PendingReview = 'pending_review';
    case Active = 'active';
    case Rejected = 'rejected';
    case Archived = 'archived';

    // Human-friendly label for display
    public function label(): string
    {
        return match ($this) {
            self::Draft => 'Draft',
            self::PendingReview => 'Pending Review',
            self::Active => 'Active',
            self::Rejected => 'Rejected',
            self::Archived => 'Archived',
        };
    }

    // Badge/tag color for UI
    public function color(): string
    {
        return match ($this) {
            self::Draft => 'gray',
            self::PendingReview => 'yellow',
            self::Active => 'green',
            self::Rejected => 'red',
            self::Archived => 'slate',
        };
    }
}
```

Usage:

```
$status = ProductStatus::Active;

$status->value; // "active"
$status->name; // "Active"
$status->label(); // "Active"
$status->color(); // "green"
```

3. Database Schema

Use a plain `string` column — **not** MySQL's native `enum()` type. The PHP enum is your single source of truth; the column just stores the string.

```
Schema::create('products', function (Blueprint $table) {  
    $table->id();  
    $table->string('product_status', 20)->default('draft');  
    // ...other columns  
    $table->timestamps();  
});
```

4. Cast the Column in the Model

Add the enum to `$casts` so Eloquent automatically converts between the DB string and the enum object:

```
<?php

namespace App\Models;

use App\Enums\ProductStatus;
use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    protected $casts = [
        'product_status' => ProductStatus::class,
    ];
}
```

Now you never work with raw strings — Eloquent hands you a real enum instance.

5. Get / Use the Data

Once cast, reading and writing "just works":

```
$product = Product::find(1);

// Reading – this is a ProductStatus instance, not a string
$product->product_status;           // ProductStatus::Active
$product->product_status->value;    // "active"
$product->product_status->label();  // "Active"

// Comparing (always compare enum-to-enum, not enum-to-string)
if ($product->product_status === ProductStatus::Active) {
    // do something
}

// Writing – assign the enum case directly
$product->product_status = ProductStatus::Archived;
$product->save();
```

Querying by status

```
// Pass the enum's value when querying
$activeProducts = Product::where('product_status', ProductStatus::Active->value)->get();

// Or pass the enum case directly – Laravel handles it too
$activeProducts = Product::where('product_status', ProductStatus::Active)->get();
```

6. Validate Incoming Requests

Use Laravel's built-in `Enum` validation rule so form input is checked against your enum's values automatically:

```
use Illuminate\Validation\Rules\Enum;
use App\Enums\ProductStatus;

$request->validate([
    'product_status' => ['required', new Enum(ProductStatus::class)],
]);
```

Tip: If someone submits `'product_status' => 'banana'`, validation fails automatically — no manual `in:` list to maintain.

Manually converting a submitted string to an enum

```
$status = ProductStatus::from($request->input('product_status'));
// throws ValueError if invalid

$status = ProductStatus::tryFrom($request->input('product_status'));
// returns null if invalid (safer for optional checks)
```

7. Loop Through All Cases (e.g. for a Select Dropdown)

Every backed enum gives you `::cases()`, which returns an array of every case — perfect for building `<select>` options.

```
@php
    use App\Enums\ProductStatus;
@endphp

<select name="product_status">
    @foreach (ProductStatus::cases() as $status)
        <option value="{{ $status->value }}"
            {{ old('product_status', $product->product_status->value ?? '') === $status->value ? 'selected' : '' }}>
            {{ $status->label() }}
        </option>
    @endforeach
</select>
```

In a Controller (pass options to the view)

```
public function edit(Product $product)
{
    return view('products.edit', [
        'product' => $product,
        'statuses' => ProductStatus::cases(),
    ]);
}
```

As a key => label array (handy for form builders / APIs)

```
$options = collect(ProductStatus::cases())
    ->mapWithKeys(fn ($status) => [$status->value => $status->label()])
    ->toArray();

// [
//     'draft' => 'Draft',
//     'pending_review' => 'Pending Review',
//     'active' => 'Active',
//     'rejected' => 'Rejected',
//     'archived' => 'Archived',
// ]
```

Use this array directly with helpers like `Form::select()` or when returning JSON to a frontend (React/Vue) for a dropdown.

8. Display Value, Label, and Color in a Blade Template

A common case: showing a product's status as a colored badge in a table or detail page.

```
{{- resources/views/products/show.blade.php -}}

<span class="badge badge-{{ $product->product_status->color() }}">
    {{ $product->product_status->label() }}
</span>

{{- Raw value, if you ever need it (e.g. for a data attribute) -}}
<span data-status="{{ $product->product_status->value }}">
    {{ $product->product_status->label() }}
</span>
```

Rendered output for a product with `product_status = active` would look like this:

Active

Example with Tailwind classes mapped from `color()`

If `color()` returns a plain word like `green`, `red`, `yellow`, map it to real Tailwind classes inside the enum itself so Blade stays clean:

```
// In ProductStatus enum
public function badgeClasses(): string
{
    return match ($this) {
        self::Draft => 'bg-gray-100 text-gray-800',
        self::PendingReview => 'bg-yellow-100 text-yellow-800',
        self::Active => 'bg-green-100 text-green-800',
        self::Rejected => 'bg-red-100 text-red-800',
        self::Archived => 'bg-slate-100 text-slate-800',
    };
}

<span class="px-2 py-1 rounded text-sm font-medium {{ $product->product_status->badgeClasses() }}">
    {{ $product->product_status->label() }}
</span>
```

Looping a table of products, showing all three together

```
<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Status</th>
    </tr>
  </thead>
  <tbody>
    @foreach ($products as $product)
      <tr>
        <td>{{ $product->name }}</td>
        <td>
          <span class="px-2 py-1 rounded text-sm {{ $product->product_status->badgeClasses() }}">
            {{ $product->product_status->label() }}
          </span>
        </td>
      </tr>
    @endforeach
  </tbody>
</table>
```

Rendered preview of the table row output:

Wireless Headphones	Active
Ceramic Mug Set	Pending Review
Old Winter Jacket	Archived
Broken Charger	Rejected
New Draft Listing	Draft

9. Quick Reference Cheat Sheet

Task	Code
Create enum	<code>php artisan make:enum Enums/ProductStatus --string</code>
Get value	<code>\$status->value</code>
Get name	<code>\$status->name</code>
From string (strict)	<code>ProductStatus::from('active')</code>
From string (safe)	<code>ProductStatus::tryFrom('active')</code>
All cases	<code>ProductStatus::cases()</code>
Validate request	<code>new Enum(ProductStatus::class)</code>
Cast in model	<code>protected \$casts = ['product_status' => ProductStatus::class];</code>
Compare	<code>\$product->product_status === ProductStatus::Active</code>

10. Apply the Same Pattern to Your Other Status Fields

Repeat steps 1–8 for each status field in your schema:

```
$ php artisan make:enum Enums/VendorVerificationStatus --string
$ php artisan make:enum Enums/VendorStatus --string
$ php artisan make:enum Enums/OrderStatus --string
$ php artisan make:enum Enums/PaymentStatus --string
$ php artisan make:enum Enums/OrderItemStatus --string
```

Each gets its own `string` column, its own `$casts` entry in its model, and its own `label()` / `color()` / `badgeClasses()` methods as needed.